

第 九 章

序列到序列模型与注意力机制

Attention Is All You Need

从 seq2seq 到大语言模型

本课目标

Learning Objectives

1

理解序列建模动机

为什么需要 seq2seq，卷积/循环编码的局限

2

掌握注意力机制

Query-Key-Value 框架与缩放点积注意力

3

吃透 Transformer 原理

自注意力、多头、残差、层归一化、位置编码

4

看清 NLP 为何需要它

并行计算 + 长程依赖 + 可扩展性

5

了解前沿进展

高效 Transformer 与大语言模型 / GPT

6

动手 PyTorch 实战

从零实现一个最小 Transformer 编码器

课程目录

Contents

01

序列到序列模型

Seq2Seq 与编码-解码架构

02

注意力机制

Attention 与自注意力 / 多头注意力

03

Transformer 模型

结构、残差、层归一化、位置编码

04

前沿一·高效 Transformer

FlashAttention · 稀疏/线性 · MoE

05

前沿二·大语言模型 / GPT

预训练 · 缩放定律 · SFT/RLHF

06

实战·PyTorch 从零实现

手写一个最小 Transformer

序列到序列模型

Sequence-to-Sequence Model · 编码-解码架构



为什么需要序列建模

变长序列与局部编码的局限

处理变长序列

神经网络处理变长向量序列时，常用卷积网络或循环网络进行编码，得到等长的输出向量序列。

但卷积/循环都是**局部编码**：

- 卷积只建模局部窗口内的依赖；
- 循环网络理论上可建模长程依赖，但随距离增大信息会衰减、难以并行。

要捕捉任意两个位置间的全局依赖，需要更强的建模方式。

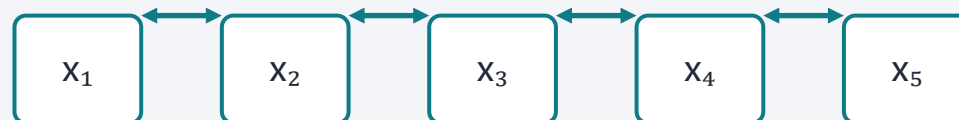
比如：

“那只昨天在公园里追着球跑的狗，它很累”

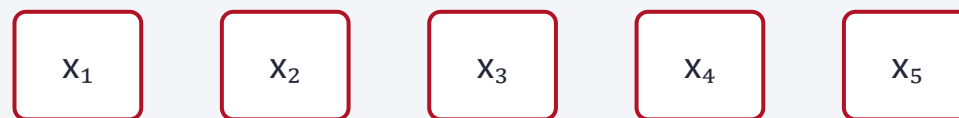
“它”指的是“狗”，中间隔了一长串。

局部编码 vs 全局依赖

局部编码：仅相邻位置交互（卷积/循环）



自注意力：任意两位置直接交互（全局）



每个位置都能“看见”整条序列

任务示例：阅读理解

从输入序列中寻找关键信息

bAbI Tasks

Facebook

20 类合成推理任务，考察机器对短文的理解与多步推理；是 seq2seq + 注意力的经典试金石。

Weston et al., 2015

SQuAD

Stanford

基于维基百科的大规模问答数据集：给定段落与问题，模型需在原文中定位答案片段。

Rajpurkar et al., 2016

核心共性

答案往往只取决于原文中的少数关键词。模型需要在长序列里“有选择地关注”相关部分——这正是注意力机制要解决的问题。

序列到序列模型

Sequence-to-Sequence, seq2seq

定义：根据给定的输入序列，通过特定模型生成另一个目标序列的方法。广泛用于机器翻译、对话、摘要、语音识别等。

1

变长输入 / 输出

输入与输出长度都不固定，且二者长度通常不相等（如翻译一句话）。

2

顺序敏感

元素的先后顺序携带语义，模型必须建模序列内部的依赖关系。

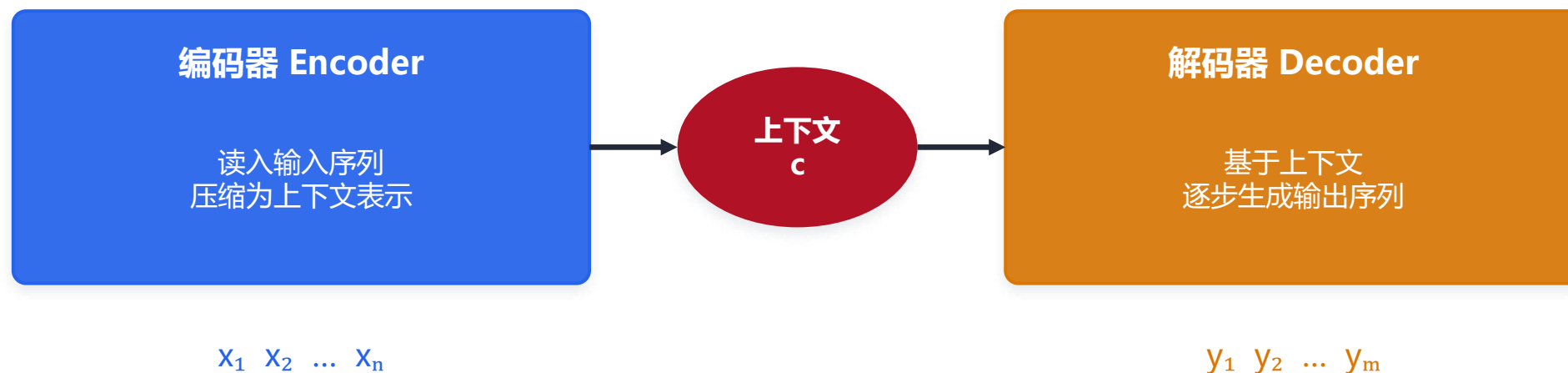
3

条件生成

输出逐步生成，每一步都依赖输入与已生成的历史。

编码-解码架构

Encoder-Decoder Architecture



关键思想：编码器把任意长度的输入压缩成定长上下文向量 c ，解码器据此自回归地生成输出。

信息瓶颈问题

定长上下文向量的局限 → 引出注意力

问题所在

- 整句话被压缩进单个定长向量 c :
 - 句子越长, c 越“记不住”早期信息;
 - 解码每一步都用同一个 c , 无法按需聚焦不同输入片段;
 - 长程依赖随距离衰减, 翻译长句质量明显下降。

解决思路: 注意力

不再依赖单一 c ——解码每一步动态地为所有输入位置分配权重,

“想看哪里就看哪里”:

- 生成每个输出词时, 重新加权组合全部输入表示;
- 权重由相关度决定, 相关的输入获得更高关注;
- 缓解长程依赖, 并带来可解释的对齐关系。

注意力机制

Attention · 从软对齐到自注意力与多头注意力



大脑中的注意力

用注意力解决信息超载

人脑每时每刻接收海量外界输入——视觉、听觉、触觉
.....

仅视觉，眼睛每秒就向视觉神经系统发送上千万比特信息。

人脑靠“注意力”解决信息超载：

- 把有限的处理资源分配给当前最相关的信息，
- 忽略大量无关输入。

神经网络中的注意力正是对这一机制的借鉴。

小实验：8 秒内找出数字 8

6	6	6	6	6	6	6	6	6
6	6	6	6	6	6	6	6	6
6	6	6	6	6	6	6	6	6
6	6	6	6	6	6	8	6	6
6	6	6	6	6	6	6	6	6
6	6	6	6	6	6	6	6	6

满屏“6”中，注意力让你迅速锁定那个“8”——这就是有选择地关注。

数学基础：向量点乘

Dot Product —— 衡量相似度

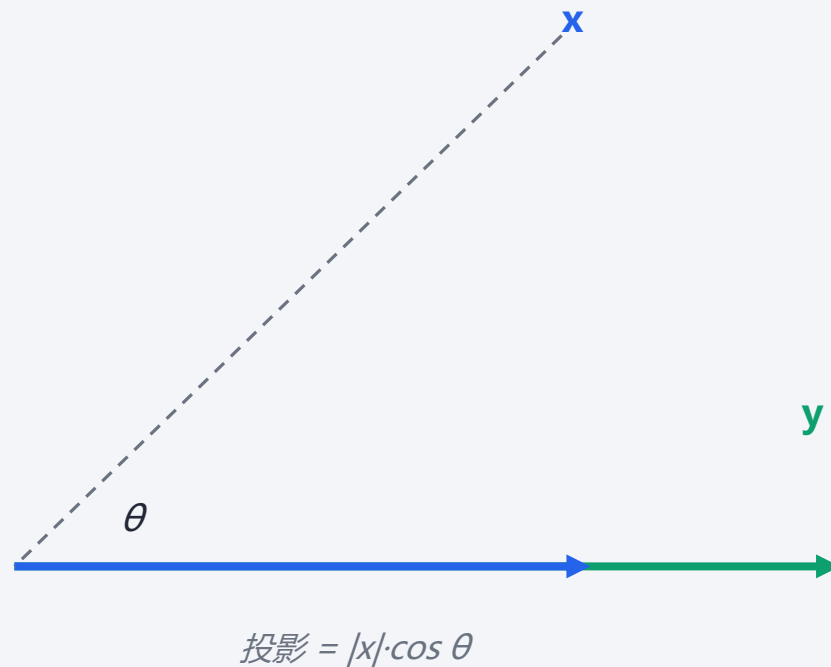
定义

$$\mathbf{x} \cdot \mathbf{y} = \sum_i x_i y_i = |\mathbf{x}| |\mathbf{y}| \cos \theta$$

几何意义：

- $\mathbf{x}$ 在 $\mathbf{y}$ 方向上的投影，再乘以 $|\mathbf{y}|$;
- 反映两向量的相似程度——
 - 点乘越大 $\rightarrow$ 方向越一致 $\rightarrow$ 越相似;
 - 点乘 $\approx 0 \rightarrow$ 近乎正交 $\rightarrow$ 几乎不相关。

注意力正是用 Query 与 Key 的点乘来打“相关度分数”。



注意力机制：通用框架

Query · Key · Value

Query 查询

当前要处理的“问题”/关注点

Key 键

每个候选信息的“索引”

Value 值

每个候选信息的“内容”

三步计算

① 打分

用 Query 与每个 Key 计算相关度 $s_i = \text{score}(q, k_i)$

② 归一化

softmax 把分数变成权重 $\alpha_i = \text{softmax}(s_i)$, $\sum \alpha_i = 1$

③ 加权求和

按权重组合所有 Value $\text{attention} = \sum_i \alpha_i v_i$

$$\text{att}(q, K, V) = \sum_i \text{softmax}_i(\text{score}(q, k_i)) \cdot v_i$$

打分函数与缩放点积

Scoring Functions

加性 / 拼接注意力

$$\text{score} = \mathbf{v}^T \tanh(W[\mathbf{q} ; \mathbf{k}])$$

早期 seq2seq (Bahdanau) 常用, 表达力强但较慢

点积注意力

$$\text{score} = \mathbf{q}^T \mathbf{k}$$

简单高效, 可高度并行; 维度高时数值偏大

缩放点积注意力

$$\text{score} = \mathbf{q}^T \mathbf{k} / \sqrt{d_k}$$

Transformer 采用: 除以 $\sqrt{d_k}$ 稳定梯度、防止 softmax 饱和

d_k 为 Key 向量维度。维度越大, 点积方差越大; $\sqrt{d_k}$ 缩放使 softmax 落在梯度敏感区。

注意力的变体

软 / 硬 / 键值对注意力

软注意力 Soft

对所有位置加权求和，权重连续、处处可导，可端到端训练。

处处可微 · 主流选择

硬注意力 Hard

只采样/选取一个位置，计算省但不可导，需强化学习等近似训练。

高效但难训练

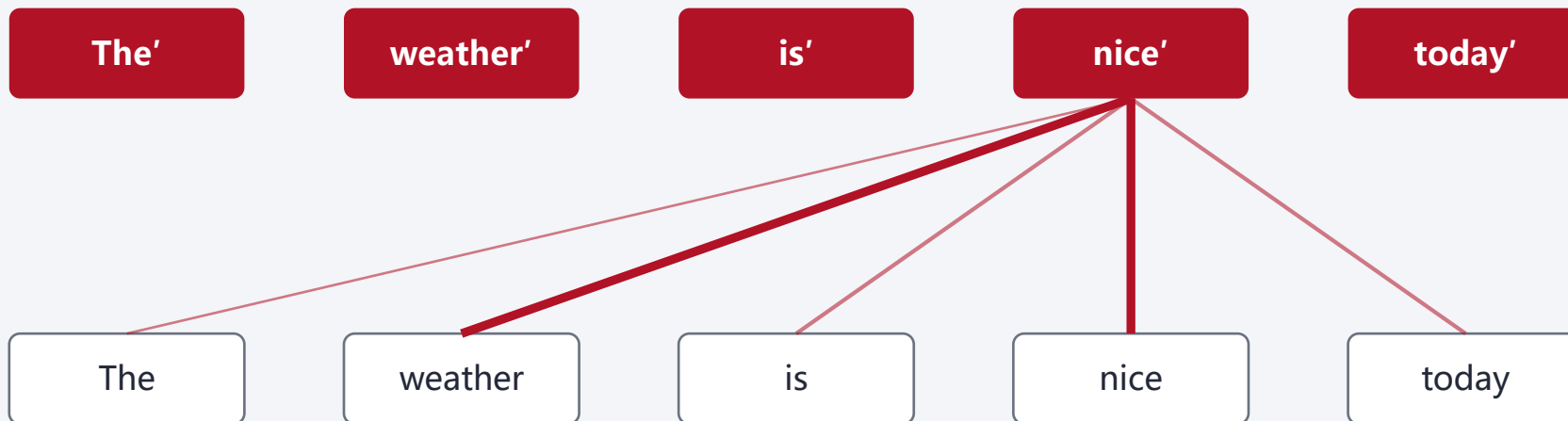
键值对注意力 KV

区分 Key 与 Value：用 Key 算权重，对 Value 加权。现代注意力的标准形式。

Q-K-V 解耦

自注意力 Self-Attention

序列内部的相互关注



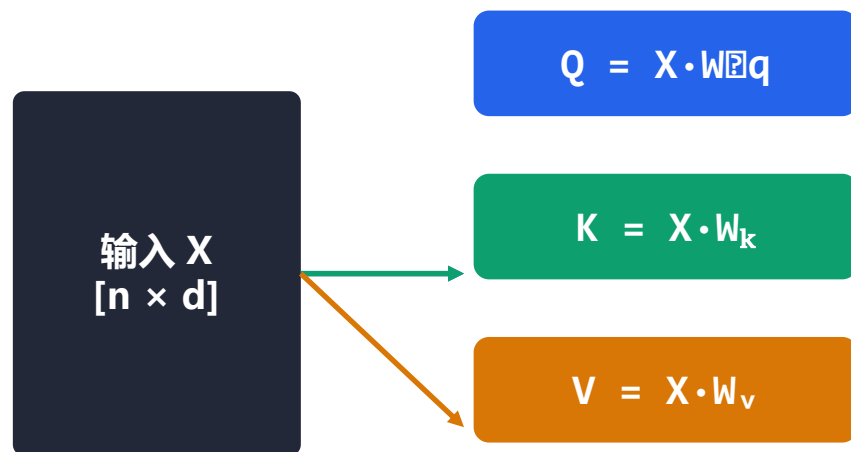
生成 "nice'" 时，模型对 weather、nice 给予更高权重（线越粗权重越大）

自注意力： Query、Key、Value 都来自同一序列。每个位置的新表示，是整条序列按相关度加权的融合——因此能一步建模任意距离的依赖，且高度并行。

QKV 模型

从输入序列到 Q / K / V

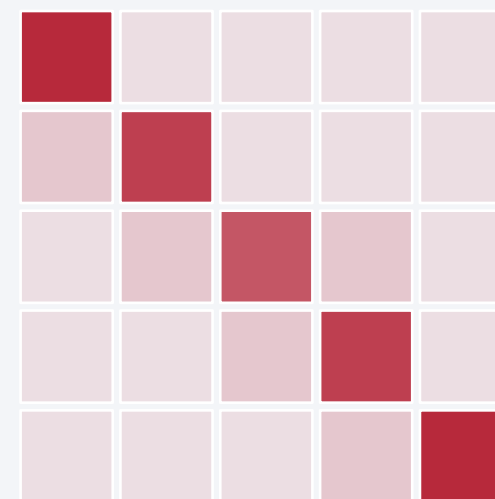
线性投影



W_q, W_k, W_v 为可学习参数矩阵

自注意力中 Q、K、V 同源于 X；交叉注意力中 Q 来自解码端，K、V 来自编码端。

注意力矩阵 $A = \text{softmax}(QK^T/\sqrt{d_k})$



行=Query, 列=Key; 颜色越深权重越大

输出 = A · V

缩放点积注意力

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

1

QK^T

所有 Query 与所有 Key
两两点积，得到 $n \times n$ 相
关度分数矩阵

2

$\div \sqrt{d_k}$

缩放，抑制高维点积过大
，稳定 softmax 梯度

3

softmax

逐行归一化为概率权重，
每行之和为 1

4

$\cdot V$

用权重对 Value 加权求和
，得到每个位置的新表示

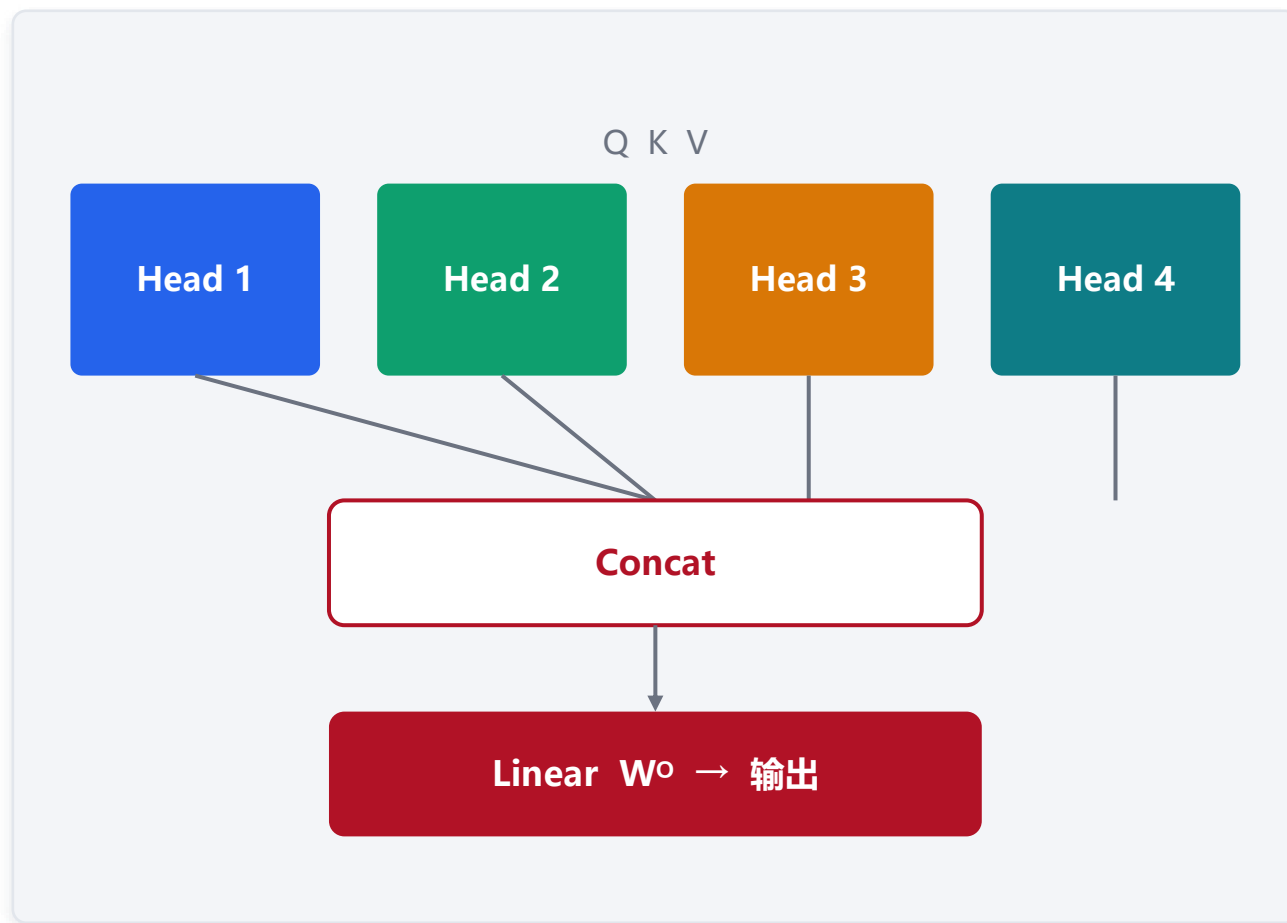
多头注意力机制

Multi-Head Attention

为什么要多头?

一句话可能同时表达多种语义关系（语法、指代、语义...）。

- 单一注意力只能关注一种模式，
 - 易被少数强相关词主导。
- 多头：多个注意力并行，
 - 各自在不同子空间从不同视角关注，
 - 再拼接 → 线性变换融合。



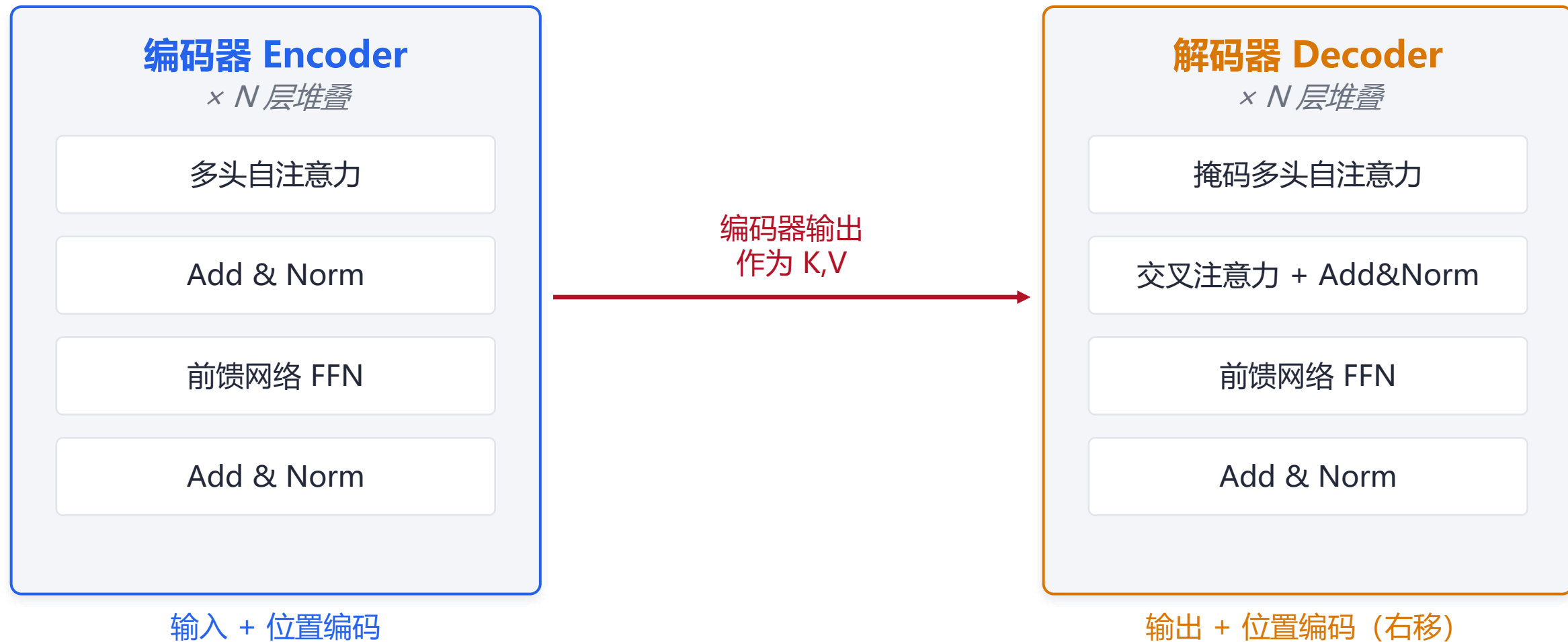
Transformer 模型

Attention Is All You Need · NeurIPS 2017



Transformer 整体架构

编码器-解码器堆叠，完全基于注意力



编码器层的两个子层

自注意力 + 前馈，皆配残差与层归一化

子层一：多头自注意力

让每个位置融合整条序列的信息，建模全局依赖。

$\text{LayerNorm}(x + \text{MultiHeadSelfAttn}(x))$

残差连接：保留原始信息、缓解梯度消失

层归一化：稳定分布、加速收敛

子层二：位置前馈网络

对每个位置独立地做两层全连接 + 非线性，增强表达力。

$\text{LayerNorm}(x + \text{FFN}(x))$

残差连接：保留原始信息、缓解梯度消失

层归一化：稳定分布、加速收敛

残差连接与层归一化

Residual Connection & Layer Normalization

残差连接 Residual

$$\text{output} = x + \text{Sublayer}(x)$$

- 把输入“抄一份”直接加到子层输出上：
 - 梯度有“高速公路”可走，缓解深层网络的梯度消失；
 - 子层只需学习“增量/修正”，更易优化；
 - 使堆叠几十层成为可能。

层归一化 LayerNorm

$$\hat{y} = \gamma \cdot (x - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta$$

- 对每个样本、在特征维度上做归一化（均值 μ 、方差 σ^2 ）；
- 不依赖 batch 大小，适合变长序列与小批量；
- γ 、 β 为可学习的缩放与平移参数；
- 稳定各层激活分布，显著加速并稳定训练。

位置前馈网络 FFN

Position-wise Feed-Forward Network

$$\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$$

逐位置独立

对序列中每个位置施加同一个全连接网络，位置之间不交互。

先升维再降维

典型 $d \rightarrow 4d \rightarrow d$ (如 $512 \rightarrow 2048 \rightarrow 512$)，中间用 ReLU/GELU 引入非线性。

互补注意力

注意力负责“跨位置混合信息”，FFN 负责“逐位置特征变换”，二者交替。

位置编码 Positional Encoding

为“无序”的注意力注入位置信息

为什么需要 PE?

自注意力对输入是“置换等变”的——打乱词序，结果只是跟着换位，模型本身看不到顺序。

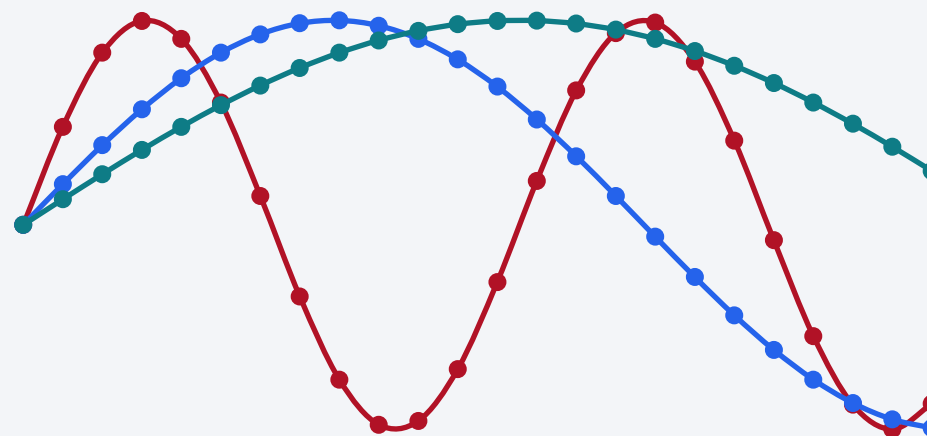
因此需把位置信息显式加到词向量上：

- 正余弦位置编码（原版）：

$$\begin{aligned} \text{PE}(\text{pos}, 2i) &= \sin(\text{pos} / 10000^{(2i/d)}) \\ \text{PE}(\text{pos}, 2i+1) &= \cos(\text{pos} / 10000^{(2i/d)}) \end{aligned}$$

不同维度用不同频率的正余弦，可表达绝对位置并外推到更长序列。

词向量 + 位置编码



不同维度 = 不同频率的正余弦波

解码器与掩码注意力

Masked Self-Attention & Cross-Attention

掩码自注意力

Masked Self-Attention

生成第 t 个词时只能看见 $1..t$ (看不到未来)。用上三角掩码把未来位置的分数置 $-\infty$ ，保证自回归与训练/推理一致。

交叉注意力

Cross-Attention

Query 来自解码器当前状态，Key/Value 来自编码器输出。让解码端“回看”整段输入——对齐源与目标。

自回归生成

Autoregressive

逐词生成：把已生成的词重新喂回解码器，预测下一个词，直到输出结束符 $\langle \text{eos} \rangle$ 。

NLP 为何需要 Transformer

对比 RNN / CNN

	RNN / LSTM	CNN	Transformer
长程依赖	随距离衰减	受感受野限制	任意两位置 $O(1)$ 直达
并行计算	时间步串行，慢	可并行	整列并行，充分利用 GPU
每层复杂度	$O(n \cdot d^2)$	$O(k \cdot n \cdot d^2)$	$O(n^2 \cdot d)$ (长序列偏高)
可扩展性	深层难训练	中等	易堆叠、随数据/参数稳定提升
可解释性	弱	弱	注意力权重可视化对齐

一句话：并行 + 长程依赖 + 可扩展，恰好契合大规模语料上的 NLP 训练需求。

为什么从 LSTM 走到 Transformer

是什么需求催生了新模型——RNN 慢、记不远、难扩展 → 注意力先救场 → 干脆只留注意力

RNN / LSTM

痛点：循环结构本身

① 串行，无法并行

第 t 步要等第 $t-1$ 步算完，GPU 并行用不上，训练慢

② 长程依赖衰减

信息逐步传递，路径 $O(n)$ ；越远越弱、梯度易消失

③ 定长上下文瓶颈

seq2seq 把整句压进一个向量 c ，句子越长越装不下

+ 注意力 Attention

先来补“瓶颈”这个洞

给 RNN 加上注意力 (Bahdanau, 2015)：

- 解码每生成一个词，都能回头“看”输入的所有位置，
- 按相关度加权，不再只依赖单一向量 c ；
- 长句翻译质量大幅提升。

关键发现：真正起作用的是注意力，那还要循环干嘛？

Transformer

只用自注意力 (2017)

丢掉 RNN，一刀解决三大痛点：

- 并行：所有位置一次算完，GPU 吃满；
- 长程：任意两词一次直连，路径 $O(1)$ ；
- 可扩展：能堆很深、随数据/参数稳定变强。

→ 催生 GPT / BERT 与大模型时代。

代价：自注意力是 $O(n^2)$ 计算量（→ 催生“高效 Transformer”），且本身看不到词序（→ 需要位置编码）。

代表模型与参考文献

- LSTM** Hochreiter & Schmidhuber, 1997 · Long Short-Term Memory
- Seq2Seq** Sutskever et al., 2014 · Sequence to Sequence Learning with Neural Networks
- 注意力 / NMT** Bahdanau et al., 2015 · Neural Machine Translation by Jointly Learning to Align and Translate
- Transformer** Vaswani et al., 2017 · Attention Is All You Need
- GPT / BERT** Radford et al., 2018 (GPT) · Devlin et al., 2019 (BERT)

前沿一 · 高效 Transformer

Efficient Transformers · 让注意力更快、更省、更长



复杂度瓶颈： $O(n^2)$

为什么长序列这么贵

自注意力要计算 $n \times n$ 的注意力矩阵：

- 时间与显存均为 $O(n^2 \cdot d)$;
- 序列翻倍，开销变 4 倍；
- 处理长文档、长对话、代码库、基因序列时迅速触顶；
- 显存（而非算力）常常成为先撞上的墙。

三条思路：① 不近似只提速 ② 稀疏/线性近似 ③ 复用缓存

开销随序列长度增长



相对开销 (对数趋势示意)

FlashAttention

IO 感知的精确注意力（不做近似）

核心思想

瓶颈不在算力，而在 GPU 显存读写（HBM↔SRAM 的搬运）。

- 分块（tiling）：把 Q/K/V 切块载入高速 SRAM；
- 在线 softmax：边算边累积，无需写出整个 $n \times n$ 矩阵；
- 重计算代替存储：反向传播时重算注意力，省显存；
- **结果与标准注意力完全一致——精确，不是近似。**

带来的收益

$O(n^2) \rightarrow O(n)$

显存 · 线性显存

2–4×

速度 · 训练/推理加速

更长

序列 · 可处理更长上下文

已成为现代大模型训练/推理的默认内核

稀疏注意力 & 线性注意力

用近似换取次二次复杂度

稀疏注意力

Sparse Attention

$O(n \cdot w)$

只让每个位置关注部分位置（局部窗口 + 少量全局/随机），把 n^2 连接稀疏化。

代表工作： Longformer · BigBird

线性注意力

Linear Attention

$O(n)$

用核函数近似 softmax，改变矩阵乘法结合顺序，把复杂度降到与长度线性相关。

代表工作： Performer · Linformer

长上下文与推理加速

位置编码演进 · KV Cache

RoPE 旋转位置编码

用旋转矩阵把相对位置注入注意力，便于外推到更长序列。当代 LLM 主流选择。

ALiBi 线性偏置

按距离对注意力分数加线性惩罚，零额外参数，训练短、推理长。

KV Cache

自回归解码时缓存历史 K/V，避免重复计算，是推理提速的关键。

GQA / MQA

多个查询头共享一组 K/V，显著减小 KV Cache 显存，加速长序列推理。

混合专家 MoE

Mixture-of-Experts · 稀疏激活，参数与算力解耦

把单个大 FFN 换成多个“专家”子网络 + 一个路由器：

- 路由器为每个 token 选少数专家（如 Top-2）；
- 只激活被选中的专家——稀疏激活；
- 总参数极大，但每个 token 的实际算力几乎不变；
- **以更低算力换取更大模型容量。**

代表：Switch Transformer、Mixtral、众多前沿大模型



前沿二 · 大语言模型 / GPT

从 Transformer 到 ChatGPT 的范式跃迁



从 Transformer 到 GPT

三种架构范式

Encoder-only

BERT 类

双向编码，擅长理解类任务（分类、抽取、检索）。

Decoder-only

GPT 类

单向自回归，逐词生成；当今大语言模型的主流范式。

Encoder-Decoder

T5 / BART 类

编码理解 + 解码生成，适合翻译、摘要等序列到序列任务。

GPT = Generative Pre-trained Transformer: 只用解码器，目标是预测“下一个词”。

GPT 家族的演进

规模与能力的阶梯（年份/参数为公开口径，部分未披露）



预训练范式

自监督的“下一个词预测”

自监督学习

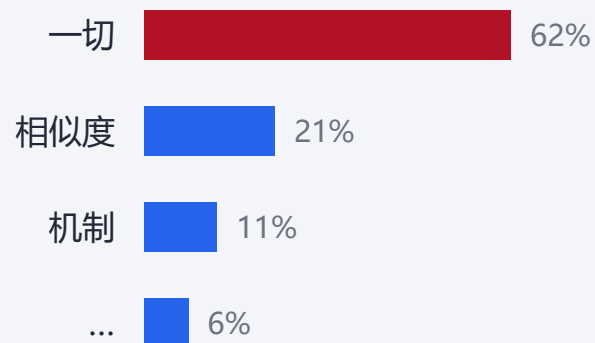
在海量无标注文本上，反复做一件简单的事：

给定前文，预测下一个 token。

- 标签来自数据本身 → 无需人工标注；
- 为预测得准，模型被迫学习语法、事实、推理与世界知识；
- 一次预训练，迁移到大量下游任务。

“知识就在大模型，参数即记忆”

下一个词预测



softmax 给出词表上的概率分布

缩放定律 Scaling Laws

损失随规模可预测地下降

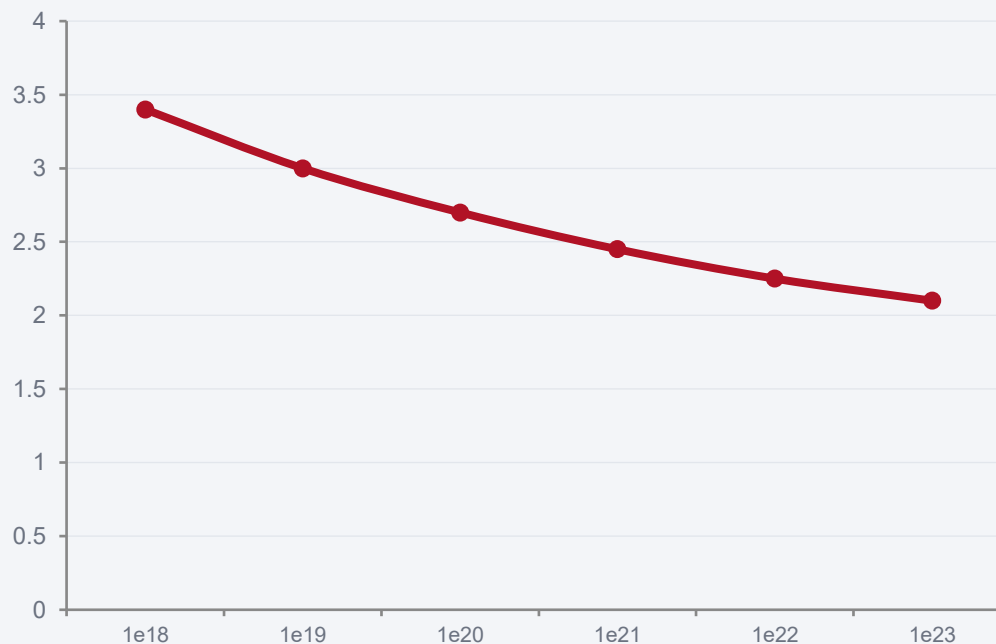
模型在测试集上的损失，随三者增长而平滑（幂律）下降：

- 参数量 N
- 数据量 D
- 计算量 C

启示：

- 能力提升在相当范围内“可预测、可外推”；
- Chinchilla 指出：参数与数据要按比例同增，
 - 算力固定时，别一味堆参数而“喂不饱”数据。

测试损失 vs 计算量



更大算力 → 更低损失（幂律，示意）

对齐：让模型听懂人话

Pretrain → SFT → RLHF

① 预训练

Pre-training

海量文本自监督，习得语言与知识。
会“续写”，但未必听话。



② 指令微调

SFT

用人工编写的“指令-示范回答”微调，教会模型按指令作答。



③ 人类反馈强化学习

RLHF

训练奖励模型刻画人类偏好，再用 RL（如 PPO）优化，使回答更有用、更安全。

RLHF 是 ChatGPT 从“会写”到“好用”的关键一步。

上下文学习与涌现能力

In-Context Learning & Emergent Abilities

1 上下文学习 ICL

在 prompt 里给几个示例 (few-shot)，模型无需更新参数即可临场学会任务。

2 思维链 CoT

提示模型“一步步想”，显著提升数学、逻辑等多步推理表现。

3 涌现能力

某些能力在模型规模跨过阈值后“突然出现”，小模型几乎为零。

4 提示工程

措辞、示例、格式、角色设定都会影响输出——用好 prompt 即用好模型。

— 实战 · PyTorch 从零实现

Build a Minimal Transformer From Scratch

6

实战蓝图

我们将从零搭起的模块

1 缩放点积注意力

```
scaled_dot_product_attention
```

注意力的最小内核

2 多头注意力

```
MultiHeadAttention
```

并行多个注意力子空间

3 位置编码

```
PositionalEncoding
```

注入序列顺序信息

4 编码器层

```
EncoderLayer
```

自注意力 + FFN + 残差 + LN

5 组装与前向

```
TransformerEncoder
```

堆叠 N 层，跑通 forward

6 对照调包

```
nn.TransformerEncoderLayer
```

与官方实现一一对应

① 缩放点积注意力

Scaled Dot-Product Attention

```
import torch, math
import torch.nn as nn
import torch.nn.functional as F

def attention(Q, K, V, mask=None):
    # Q,K,V: (B, heads, T, d_k)
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)
    attn = F.softmax(scores, dim=-1)
    return attn @ V, attn
```

讲解

对应公式 $\text{softmax}(QK^T/\sqrt{d_k}) \cdot V$ 。

- @ 是批量矩阵乘法;
- transpose 交换最后两维做 K^T ;
- masked_fill 把被遮挡位置置 $-\infty$, softmax 后 ≈ 0 ;
- 返回输出与注意力权重 (便于可视化)。

② 多头注意力

Multi-Head Attention

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.d_k = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, d_model*3)
        self.out = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        B, T, _ = x.shape
        qkv = self.qkv(x).reshape(B, T, 3, self.h, self.d_k)
        q, k, v = qkv.permute(2, 0, 3, 1, 4)
        o, _ = attention(q, k, v, mask) # 见 ①
        o = o.transpose(1, 2).reshape(B, T, -1)
        return self.out(o)
```

讲解

- 一个线性层同时产出 Q/K/V ($d_{\text{model}} \times 3$) 再拆分;
- reshape/permute 把维度切成 h 个头;
- 每个头在 d_k 子空间独立做注意力;
- 拼回后用 self.out 线性融合 (即 W^o)。

③ 位置编码

Sinusoidal Positional Encoding

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(max_len).unsqueeze(1)
        div = torch.exp(torch.arange(0, d_model, 2) *
                        -(math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]
```

讲解

- 偶数维用 sin、奇数维用 cos;
- div 控制不同维度的频率;
- register_buffer: 随模型保存但不参与训练;
- forward 把位置编码直接加到词向量上。

现代 LLM 多改用 RoPE / ALiBi (见前沿章)。

④ 编码器层

Self-Attention + FFN + 残差 + LayerNorm

```
class EncoderLayer(nn.Module):
    def __init__(self, d_model, n_heads, d_ff, p=0.1):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, n_heads)
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.ReLU(),
            nn.Linear(d_ff, d_model))
        self.n1, self.n2 = nn.LayerNorm(d_model), nn.LayerNorm(d_model)
        self.drop = nn.Dropout(p)

    def forward(self, x, mask=None):
        x = self.n1(x + self.drop(self.attn(x, mask))) # 残差+LN
        x = self.n2(x + self.drop(self.ff(x)))         # 残差+LN
        return x
```

讲解

两个子层，结构完全对称：

LayerNorm(x + Sublayer(x))

- 子层一：多头自注意力；
- 子层二：前馈网络 FFN；
- 残差 + Dropout + LayerNorm 保证可堆深、易训练。

⑤ 组装与前向

堆叠 N 层，跑通一次 forward

```
class TransformerEncoder(nn.Module):
    def __init__(self, vocab, d_model=256, N=6,
                  n_heads=8, d_ff=1024):
        super().__init__()
        self.emb = nn.Embedding(vocab, d_model)
        self.pe = PositionalEncoding(d_model)
        self.layers = nn.ModuleList(
            [EncoderLayer(d_model, n_heads, d_ff) for _ in range(N)])

    def forward(self, ids, mask=None):
        x = self.pe(self.emb(ids))
        for layer in self.layers: x = layer(x, mask)
        return x          # (B, T, d_model)

model = TransformerEncoder(vocab=10000)
y = model(torch.randint(0, 10000, (2, 20)))
```

讲解

- Embedding + 位置编码 → 进入层堆叠;
- ModuleList 堆叠 N=6 个编码器层;
- 输出可接分类头、CRF 或解码器;
- 在上面加交叉熵损失 + Adam 即可训练。

对照官方实现与延伸

理解原理后，工程上一行调包

```
# PyTorch 内置，与我们手写的——对应
layer = nn.TransformerEncoderLayer(
    d_model=256, nhead=8, dim_feedforward=1024)
enc = nn.TransformerEncoder(layer, num_layers=6)

# HuggingFace: 直接加载预训练模型
from transformers import AutoModel
m = AutoModel.from_pretrained('bert-base')
```

为什么还要手写？

调包很快，但只有亲手实现，才能真正看懂维度怎么流动、mask 如何起效、残差与 LN 放在哪里——出问题时才调得动。

延伸练习

- 补上解码器与掩码自注意力，做一个英↔中翻译玩具模型；
- 把正余弦位置编码替换成 RoPE，对比长序列外推；
- 接入 FlashAttention 内核，测一测显存与吞吐；
- 基于 HuggingFace 在 SQuAD / 文本分类上微调，跑通完整流程。

本章小结

Summary

序列到序列

编码-解码把变长输入映射为变长输出；定长上下文的瓶颈催生注意力。

注意力机制

Query-Key-Value + 缩放点积；自注意力一步建模全局依赖，多头多视角。

Transformer

自注意力 + FFN + 残差 + 层归一化 + 位置编码，完全并行、可深可扩展。

前沿进展

高效 Transformer (FlashAttention/稀疏/MoE) 与大语言模型/GPT (预训练·对齐·涌现)。

谢谢

Attention Is All You Need

欢迎在课后动手实现你的第一个 Transformer